

Explaining the RNN Screenshot

Why Hidden State Is Memory, and Why the Lower Formula Is Not

April 9, 2026

Contents

1	Why the Screenshot Introduces Hidden State	1
2	How the Recurrent Part Works	1
3	The Lower Formula, Slowly Explained	2
3.1	What Each Symbol Means	2
3.2	Step 1: From Input Matrix to Hidden Features	2
3.3	Step 2: From Hidden Features to Output	3
3.4	Why This Formula Has No Memory	3
4	A Concrete Numerical Example	3
5	Summary	4

1 Why the Screenshot Introduces Hidden State

The screenshot is trying to solve a practical problem, not invent a mysterious symbol. In sequence tasks, the current token rarely makes sense by itself. The word at time t may depend on something said several steps earlier. If a model stores the entire past literally, the storage burden and the parameter burden both grow quickly.

That is why the screenshot writes

$$P(x_t \mid x_{t-1}, \dots, x_1) \approx P(x_t \mid h_{t-1}).$$

The point is simple: replace a long history by a compact summary h_{t-1} . The hidden state is therefore a *working memory*. It is not the full past. It is a compressed note about the past that is good enough for the next decision.

2 How the Recurrent Part Works

The recurrent rule

$$h_t = f(x_t, h_{t-1})$$

means that each step uses two inputs:

- the new observation x_t ,

- the previous memory h_{t-1} .

The output is a new memory h_t . This is the essential difference from an ordinary feedforward layer. The model does not restart from zero at every step. It carries forward a state.

One may think of h_t as a notebook that is rewritten one line at a time. The new token writes new information into the notebook, but the old page still influences what remains.

3 The Lower Formula, Slowly Explained

The lower half of the screenshot writes

$$H = \phi(XW_{xh} + b_h), \quad O = HW_{hq} + b_q.$$

This part is not yet a recurrent model. It is a standard feedforward map. To see that clearly, it helps to unpack every symbol.

3.1 What Each Symbol Means

Assume:

- $X \in \mathbb{R}^{n \times d}$: a batch of inputs,
- n : batch size,
- d : number of input features per example,
- $W_{xh} \in \mathbb{R}^{d \times h}$: weights from input to hidden features,
- $b_h \in \mathbb{R}^{1 \times h}$: hidden-layer bias,
- $H \in \mathbb{R}^{n \times h}$: hidden representation,
- $W_{hq} \in \mathbb{R}^{h \times q}$: weights from hidden features to output,
- $b_q \in \mathbb{R}^{1 \times q}$: output-layer bias,
- $O \in \mathbb{R}^{n \times q}$: output scores.

So the lower formula says nothing more exotic than:

turn each input row into hidden features, then turn those hidden features into outputs.

3.2 Step 1: From Input Matrix to Hidden Features

The expression

$$H = \phi(XW_{xh} + b_h)$$

contains three small operations.

First, linear mixing. Multiply X by W_{xh} . Each input row with d numbers becomes a row with h numbers. This is just feature recombination.

Second, add bias. Add b_h . The same bias row is added to every sample in the batch. Bias lets each hidden unit shift its default response.

Third, apply nonlinearity. Apply ϕ elementwise, such as ReLU or tanh. This makes the layer more expressive than a single linear map.

After these three steps, H is a hidden representation. It is called “hidden” only because it is an internal feature layer. It is *not* a time-carrying memory state.

3.3 Step 2: From Hidden Features to Output

The second formula

$$O = HW_{hq} + b_q$$

maps the hidden features to output scores.

If this is classification, each row of O is often a vector of logits, and one may later apply softmax. If this is regression, O may already be the final prediction. In either case, this is still an ordinary one-shot mapping:

input $\rightarrow$ hidden features $\rightarrow$ output.

3.4 Why This Formula Has No Memory

The crucial fact is what the formula does *not* contain. It has no term such as h_{t-1} . That means:

- it does not explicitly carry information from the previous time step,
- it does not update a running state,
- it treats each input row as a fresh computation.

So even though the letter H appears, this is still not sequence memory in the RNN sense. It is just a hidden layer in a feedforward network.

This is why the screenshot contrasts the two halves:

- the upper half says, “keep a memory state and update it”;
- the lower half says, “apply a static feature map to the current input”.

4 A Concrete Numerical Example

Take one input row

$$x = [2, 1],$$

and suppose

$$W_{xh} = \begin{bmatrix} 1 & -1 \\ 2 & 1 \end{bmatrix}, \quad b_h = [0, 1].$$

Choose ϕ to be ReLU.

Then

$$xW_{xh} = [2, 1] \begin{bmatrix} 1 & -1 \\ 2 & 1 \end{bmatrix} = [4, -1].$$

After adding bias,

$$[4, -1] + [0, 1] = [4, 0].$$

After ReLU,

$$H = [4, 0].$$

Now let

$$W_{hq} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}, \quad b_q = [0].$$

Then

$$O = HW_{hq} + b_q = [4, 0] \begin{bmatrix} 1 \\ -1 \end{bmatrix} = 4.$$

This example shows the lower formula very plainly. It only transforms the current input into an output. If we want memory of an earlier word such as `not`, the formula itself has no dedicated place to store it. That is exactly why the upper recurrent formula introduces h_{t-1} .

5 Summary

The screenshot is contrasting two ideas:

- RNN: keep a rolling memory h_t so the past can influence the present,
- feedforward network: compute hidden features H and outputs O from the current input only.

So the lower formula is not difficult because of the symbols. It is only the standard pipeline

$$\text{input} \rightarrow \text{hidden layer} \rightarrow \text{output layer}.$$

Its limitation is also standard: by itself, it does not remember earlier time steps.